Codeforces Round 993 (Div. 4) 个人题解（A-H）

所有翻译均为机翻！

A.Easy Problem

time limit per test: 1 second

memory limit per test: 256 megabytes

Difficulty:800

原题

Cube is given an integer n. She wants to know how many ordered pairs of positive integers (a, b) there are such that $a = n - b$. Since Cube is not very good at math, please help her!

给 Cube 一个整数 n 。她想知道有多少对有序正整数 (a, b) 令 $a = n - b$ 成对。由于 Cube 不擅长数学，请帮助她！

Input

The first line contains an integer t ($1 \leq t \leq 99$) — the number of test cases.

The only line of each test case contains an integer n ($2 \leq n \leq 100$).

第一行包含一个整数 t ($1 \leq t \leq 99$) - 测试用例的数量。

每个测试用例的唯一一行包含一个整数 n ($2 \leq n \leq 100$)。

Output

For each test case, output the number of ordered pairs (a, b) on a new line.

对于每个测试用例，另起一行输出有序对 (a, b) 的数量。

Example

Input
3
2
4
6

Output
1
3
5

题解

a的取值范围为$[1, n]$， 有$n - 1$个取值， 每个取值都有对应的b满足$a = n - b$

所以答案为$n - 1$

代码如下

```
t = int(input())
for _ in range(t):
    n = int(input())
    print(n-1)
```

B. Normal Problem

time limit per test: 1 second

memory limit per test: 256 megabytes

Difficulty:800

原题

A string consisting of only characters 'p', 'q', and 'w' is painted on a glass window of a store. Ship walks past the store, standing directly in front of the glass window, and observes string a. Ship then heads inside the store, looks directly at the same glass window, and observes string b.

Ship gives you string a. Your job is to find and output b.

一家商店的玻璃橱窗上画着一串只有字符 "p"、"q "和 "w "的字符串。Ship 走过商店，站在玻璃橱窗正前方，观察到字符串 a 。然后，小船走进商店，直视同一玻璃橱窗，观察到字符串 b 。

飞船给出了字符串 a 。您的任务是找到并输出 b 。

一家商店的玻璃橱窗上画着一个字符串，字符串只由字符 "p"、"q "和 "w "组成。Ship 走过商店，站在玻璃窗正前方，观察到字符串 a 。然后 Ship 走进商店，直视同一个玻璃橱窗，观察到字符串 b 。飞船给出了字符串 a 。您的任务是找到并输出 b 。

Input

The first line contains an integer t ($1 \le t \le 100$) — the number of test cases.

The only line of each test case contains a string a ($1 \le |a| \le 100$) — the string Ship observes from outside the store. It is guaranteed that a only contains characters 'p', 'q', and 'w'.

第一行包含一个整数 t ($1 \le t \le 100$) - 测试用例的数量。

每个测试用例的唯一一行包含一个字符串 a ($1 \le |a| \le 100$) - Ship 从商店外部观察到的字符串。保证 a 只包含字符 "p"、"q "和 "w"。

Output

For each test case, output string b, the string Ship observes from inside the store, on a new line.

对于每个测试用例，在新的一行中输出字符串 b ，即 Ship 在商店内部观察到的字符串。

Example

Input
5
qwq
ppppp
pppwwwqqq
wqpqwpqwwqp
pqpqpqpq

Output
pwp
qqqqq
pppwwwqqq
qpwwpqwpqpw
pqpqpqpq

题解

从后往前遍历字符串，将p改为q， q改为p即可。

代码如下。

```
t = int(input())
for _ in range(t):
    s = input()
    s = s[::-1]
    for i in s:
        if i=='p':
            print('q', end='')
        elif i=='q':
            print('p', end='')
        else:
            print(i, end='')
    print()
```

C. Hard Problem

time limit per test: 1 second

memory limit per test: 256 megabytes

Difficulty:800

原题

Ball is the teacher in Paperfold University. The seats of his classroom are arranged in 2 rows with m seats each.

Ball is teaching $a+b+c$ monkeys, and he wants to assign as many monkeys to a seat as possible. Ball knows that a of them only want to sit in row 1, b of them only want to sit in row 2, and c of them have no preference. Only one monkey may sit in each seat, and each monkey's preference must be followed if it is seated.

What is the maximum number of monkeys that Ball can seat?

波尔是纸折大学的老师。他教室里的座位排成 2 行，每行有 m 个座位。

波尔正在给 $a+b+c$ 只猴子上课，他想给尽可能多的猴子分配座位。波尔知道 a 只想坐在 1 排， b 只想坐在 2 排， c 没有任何偏好。每个座位只能坐一只猴子，而且每只猴子都必须按照自己的偏好入座。

波尔最多可以让多少只猴子坐下?

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

Each test case contains four integers m, a, b, and c ($1 \le m, a, b, c \le 10^8$).
第一行包含一个整数 t ($1 \le t \le 10^4$) - 测试用例的数量。

每个测试用例包含四个整数 m 、 a 、 b 和 c ($1 \le m, a, b, c \le 10^8$)。

Output

For each test case, output the maximum number of monkeys you can seat.

对于每个测试用例，输出可容纳猴子的最大数量。

Example

Input
5
10 5 5 10
3 6 1 1
15 14 12 4
1 1 1 1
420 6 9 69

Output
5
10 5 5 10
3 6 1 1
15 14 12 4
1 1 1 1
420 6 9 69

题解

模拟即可。
代码如下。

```
t = int(input())
for _ in range(t):
    m, a, b, c = map(int, input().split())
    u1 = min(a, m)
    u2 = min(b, m)
    t1 = min(c, m-u1)
    c -= t1
    u1 += t1
    t2 = min(c, m-u2)
    c -= t2
    u2 += t2
    print(u1+u2)
```

D. Harder Problem

time limit per test: 2 second

memory limit per test: 256 megabytes

Difficulty:1000

原题

Given a sequence of positive integers, a positive integer is called a mode of the sequence if it occurs the maximum number of times that any positive integer occurs. For example, the mode of $[2,2,3]$ is 2. Any of 9, 8, or 7 can be considered to be a mode of the sequence $[9,9,8,8,7,7]$.

You gave UFO an array a of length n. To thank you, UFO decides to construct another array b of length n such that a_i is a mode of the sequence $[b_1,b_2,\ldots,b_i]$ for all $1 \le i \le n$.

However, UFO doesn't know how to construct array b, so you must help her. Note that $1 \le b_i \le n$ must hold for your array for all $1 \le i \le n$.

给定一个正整数序列，如果一个正整数出现的次数是所有正整数出现的最大次数，那么这个正整数就称为序列的模。例如，$[2,2,3]$ 的模是 2 。9 、8 或 7 中的任何一个都可以视为序列 $[9,9,8,8,7,7]$ 的模。

你给了 UFO 一个长度为 n 的数组 a 。为了感谢你，UFO 决定再构造一个长度为 n 的数组 b ，使得所有 $1 \le i \le n$ 的数组 a_i 都是数列 $[b_1,b_2,\ldots,b_i]$ 的模。

然而，UFO 不知道如何构造数组 b ，所以你必须帮助她。请注意，在所有 $1 \le i \le n$ 中，你的数组 $1 \le b_i \le n$ 必须成立。

Input

The first line contains t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \le n \le 2 \cdot 10^5$) — the length of a.

The following line of each test case contains n integers $a_1,a_2,\ldots,a_n$ ($1 \le a_i \le n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

第一行包含一个整数 t ($1 \le t \le 10^4$) - 测试用例的数量。

每个测试用例包含四个整数 m 、 a 、 b 和 c ($1 \le m,a,b,c \le 10^8$)。

第一行包含 t ($1 \le t \le 10^4$) - 测试用例数。

每个测试用例的第一行包含一个整数 n ($1 \le n \le 2 \cdot 10^5$) - a 的长度。

每个测试用例的下一行包含 n 个整数 $a_1,a_2,\ldots,a_n$ ($1 \le a_i \le n$)。

保证所有测试用例中 n 的总和不超过 $2 \cdot 10^5$ 。

Output

For each test case, output n numbers $b_1,b_2,\ldots,b_n$ ($1 \le b_i \le n$) on a new line. It can be shown that b can always be constructed. If there are multiple possible arrays, you may print any.

对于每个测试用例，在新行中输出 n 数字 $b_1,b_2,\ldots,b_n$ ($1 \le b_i \le n$)。可以证明 b 总是可以构造出来的。如果有多个可能的数组，可以打印任意一个。

Example

Input
4
2
1 2
4

1 1 1 2
8
4 5 5 5 1 1 2 1
10
1 1 2 2 1 1 3 3 1 1

Output
1 2
1 1 2 2
4 5 5 1 1 2 2 3
1 8 2 2 1 3 3 9 1 1

题解

如果你在使用python回答此题， 请不要使用set!!!!!（被构造哈希冲突了）

对于一个包含$[1, 2, 3, ..., n]$的数组， 显然其中任何一个元素都可以作为数组的模。

我们只需要保证a中新出现的元素，b中已经出现了即可。

使用一个数组代表数字有没有使用过，另一个代表数字有没有出现。

代码如下。

```
t = int(input())
for _ in range(t):
    n = int(input())
    a = list(map(int, input().split()))
    vis = [0 for i in range(n+1)]
    m = [0 for i in range(n+1)]
    idx = 1
    for i in a:
        vis[i] = 1
    ans = [0 for i in range(n)]
    for i in range(n):
        if m[a[i]] == 0:
            ans[i] = a[i]
            m[a[i]] = 1
        else:
            while vis[idx]:
                idx += 1
            vis[idx] = 1
            ans[i] = idx
    print(*ans)
```

E.Insane Problem

time limit per test: 2 second

memory limit per test: 256 megabytes

Difficulty:1300

原题

Wave is given five integers k, l_1, r_1, l_2, and r_2. Wave wants you to help her count the number of ordered pairs (x, y) such that all of the following are satisfied:

- $l_1 \le x \le r_1$.
- $l_2 \le y \le r_2$.
- There exists a non-negative integer n such that $\frac{y}{x} = k^n$.

小波得到五个整数 k ， l_1 ， r_1 ， l_2 和 r_2 。小波想让你帮她计算出满足以下所有条件的有序数对 (x, y) 的个数：

- $l_1 \le x \le r_1$.
- $l_2 \le y \le r_2$.
- 存在一个非负整数 n ，使得 $\frac{y}{x} = k^n$.

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The only line of each test case contains five integers k, l_1, r_1, l_2, and r_2 ($2 \le k \le 10^9, 1 \le l_1 \le r_1 \le 10^9, 1 \le l_2 \le r_2 \le 10^9$).

第一行包含一个整数 t ($1 \le t \le 10^4$) - 测试用例的数量。

每个测试用例的唯一一行包含五个整数 k 、 l_1 、 r_1 、 l_2 和 r_2 ($2 \le k \le 10^9, 1 \le l_1 \le r_1 \le 10^9, 1 \le l_2 \le r_2 \le 10^9$)。

Output

For each test case, output the number of matching ordered pairs (x, y) on a new line.

对于每个测试用例，在新行中输出匹配有序对 (x, y) 的数量。

Example

Input
5
2 2 6 2 12
2 1 1000000000 1 1000000000
3 5 7 15 63
1000000000 1 5 6 1000000000
15 17 78 2596 20914861

Output
12
1999999987
6
1
197

题解

$[l,r]$的范围很大，对其中每个数进行枚举不现实。

考虑对$k^n(k\neq 1)$的范围($k=1$时只有(a,a)这样的对，答案为两个区间的重叠部分)
当$k=2$时，就算$r=10^9$， n最大取到29($2^{29}=536870912$，$2^{30}=1073741824>10^9$)。
可以看到，如果我们对n进行枚举，考虑每个n的取值下有多少对，其消耗的时间是可以接受的。

$\because \frac{y}{x}=k^n$
$\therefore y=k^n\cdot x$
$\because l_2\leq y\leq r_2$
$\therefore \left\lceil\frac{l_2}{k^n}\right\rceil\leq x\leq\left\lfloor\frac{r_2}{k^n}\right\rfloor$
$\because l_1\leq x\leq r_1$
$\therefore \max\left(l_1,\left\lceil\frac{l_2}{k^n}\right\rceil\right)\leq x\leq\min\left(r_1,\left\lfloor\frac{r_2}{k^n}\right\rfloor\right)$

这就是每个n对应的x的范围，当区间长度小于0是取0.

代码如下

```
import math

t = int(input())
for _ in range(t):
    k, l1, r1, l2, r2 = map(int, input().split())
    count = 0
    if k == 1:
        print(max(min(r2, r1) - max(l1, l2), 0))
    else:
        for n in range(int(math.log(r2, k)) + 1):
            lower_x = max(l1, math.ceil(l2 / (k**n)))
            upper_x = min(r1, math.floor(r2 / (k**n)))
            if lower_x <= upper_x:
                count += upper_x - lower_x + 1
        print(count)
```

F. Easy Demon Problem

time limit per test: 4 second

memory limit per test: 256 megabytes

Difficulty:1900

原题

For an arbitrary grid, Robot defines its beauty to be the sum of elements in the grid.

Robot gives you an array a of length n and an array b of length m. You construct a n by m grid M such that $M_{i,j}=a_i\cdot b_j$ for all $1\leq i\leq n$ and $1\leq j\leq m$.

Then, Robot gives you q queries, each consisting of a single integer x. For each query, determine whether or not it is possible to perform the following operation **exactly** once so that M has a beauty of x:

1. Choose integers r and c such that $1\leq r\leq n$ and $1\leq c\leq m$

2. Set $M_{i,j}$ to be 0 for all ordered pairs (i, j) such that $i = r$, $j = c$, or both.

Note that queries are **not persistent**, meaning that you do not actually set any elements to 0 in the process — you are only required to output if it is possible to find r and c such that if the above operation is performed, the beauty of the grid will be x. Also, note that you must perform the operation for each query, even if the beauty of the original grid is already x.

对于任意网格，Robot 将其美感定义为网格中元素的总和。

机器人会给你一个长度为 n 的数组 a 和一个长度为 m 的数组 b 。你构造了一个 n 乘 m 的网格 M ，使得所有 $1 \le i \le n$ 和 $1 \le j \le m$ 都满足 $M_{i,j} = a_i \cdot b_j$ 。

然后，机器人会给出 q 个查询，每个查询都包含一个整数 x 。对于每个查询，请判断是否可以***精确地执行一次下面的操作，从而使 M 具有 x 的美感：

1. 选择整数 r 和 c ，使得 $1 \le r \le n$ 和 $1 \le c \le m$ 为整数。
2. 设置 $M_{i,j}$ 为 0 ，$i = r$ 或 $j = c$。

请注意，查询是**不持久的**，也就是说，在查询过程中，您不会将任何元素设置为 0 --您只需要输出是否可以找到 r 和 c ，如果执行了上述操作，网格的美观度将为 x 。此外，请注意，即使原始网格的美观度已经是 x ，您也必须对每个查询执行上述操作。

Input

The first line contains three integers n, m, and q ($1 \le n, m \le 2 \cdot 10^5, 1 \le q \le 5 \cdot 10^4$) — the length of a, the length of b, and the number of queries respectively.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($0 \le |a_i| \le n$).

The third line contains m integers $b_1, b_2, \ldots, b_m$ ($0 \le |b_i| \le m$).

The following q lines each contain a single integer x ($1 \le |x| \le 2 \cdot 10^5$), the beauty of the grid you wish to achieve by setting all elements in a row and a column to 0.

第一行包含三个整数 n 、 m 和 q ($1 \le n, m \le 2 \cdot 10^5, 1 \le q \le 5 \cdot 10^4$) - 分别是 a 的长度、 b 的长度和查询次数。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$ ($0 \le |a_i| \le n$)。

第三行包含 m 个整数 $b_1, b_2, \ldots, b_m$ ($0 \le |b_i| \le m$)。

下面的 q 行中各包含一个整数 x ($1 \le |x| \le 2 \cdot 10^5$)，即通过将一行和一列中的所有元素都设置为 0 所获得的网格美感。

Output

For each testcase, output "YES" (without quotes) if there is a way to perform the aforementioned operation such that the beauty is x, and "NO" (without quotes) otherwise.

You can output "YES" and "NO" in any case (for example, strings "yES", "yes" and "Yes" will be recognized as a positive response).

对于每个测试用例，如果有方法执行上述操作，使美观度为x，则输出“YES”（不带引号），否则为“NO”（无引号）。

在任何情况下，您都可以输出“是”和“否”（例如，字符串“YES”、“YES”和“YES”将被识别为正确答案）。

Example

Input
3 3 6
-2 3 -3
-2 2 -1
-1
1
-2
2
-3
3

Output
NO
YES
NO
NO
YES
NO

Input
5 5 6
1 -2 3 0 0
0 -2 5 0 -3
4
-3
5
2
-1
2

Output
YES
YES
YES
YES
NO
YES

题解

考虑去掉某一列后元素总和如何计算。（显然不能暴力计算）

设初始状态下元素总和为S, 有

$$
\begin{aligned}
s &= a_1 \cdot b_1 + a_1 \cdot b_2 + \cdots + a_1 \cdot b_m + a_2 \cdot b_1 + a_2 \cdot b_2 + \cdots + a_2 \cdot b_m + \cdots + a_n \cdot b_n \\
&= a_1 * \sum_{i=1}^{m} b_i + a_2 * \sum_{i=1}^{m} b_i + \cdots + a_n * \sum_{i=1}^{m} b_i \\
&= \sum_{i=1}^{n} a_i * \sum_{i=1}^{m} b_i
\end{aligned}
$$

即a, b数组元素和的积

去掉第i行，第j列后，元素和为
$s^{'} = s - a_i \cdot \sum_{k=1}^{m} b_k - b_j \cdot \sum_{k=1}^{m} a_k + a_i \cdot b_j$
$= (\sum_{k=1}^{n} a_k - a_i) * (\sum_{k=1}^{m} b_k - b_j)$

我们只需遍历x的因子对(x, y)，观察a中是否有$\sum_{i=1}^{n} a_i - x$以及b中是否有$\sum_{i=1}^{n} b_i - y$即可

代码如下

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

const ll maxn = 200009;
vector<ll> pa(2*maxn+1, 0), pb(2*maxn+1, 0);// 模仿哈希表（也被构造哈希攻击了）
int main(){
    ll n = read(), m = read(), q = read();
    ll sa=0, sb = 0, t;
    for(ll i=0;i<n;i++){
        t = read();
        sa += t;
        pa[t+maxn] = 1;
    }
    for(ll i=0;i<m;i++){
        t = read();
        sb += t;
        pb[t+maxn] = 1;
    }
    auto check = [&](ll x, ll y) {
        ll na = sa - x + maxn;
        ll nb = sb - y + maxn;
        return (0 <= na && na < 2*maxn+1 && pa[na]&&0 <= nb && nb < 2*maxn+1 &&
pb[nb]);
    };
    ll f, x, a, b;
    while(q--){
        f = 0;
        x = read();
        for(ll i=1;i<=sqrt(abs(x))+1;i++){
            if (x%i==0){
                a = i;
                b = x/i;
                if(check(a, b)||check(b, a)||check(-a, -b)||check(-b, -a)){
                    f = 1;
                    break;
                }
            }
        }
        if (f) puts("YES");
```

```
        else puts("NO");
    }
}
```

G1. Medium Demon Problem (easy version)

time limit per test: 2 second

memory limit per test: 256 megabytes

Difficulty:1700

原题

This is the easy version of the problem. The key difference between the two versions is highlighted in bold.

A group of n spiders has come together to exchange plushies. Initially, each spider has 1 plushie. Every year, if spider i has at least one plushie, he will give exactly one plushie to spider r_i. Otherwise, he will do nothing. Note that all plushie transfers happen at the same time. **In this version, if any spider has more than 1 plushie at any point in time, they will throw all but 1 away.**

The process is stable in the current year if each spider has the same number of plushies (before the current year's exchange) as he did the previous year (before the previous year's exchange). Note that year 1 can never be stable.

Find the first year in which the process becomes stable.

这是问题的简单版本。两个版本的主要区别以粗体标出。

一群 n 蜘蛛聚在一起交换毛绒玩具。最初，每只蜘蛛都有 1 个毛绒玩具。每年，如果蜘蛛 i 至少有一个毛绒玩具，他就会给蜘蛛 r_i 一个毛绒玩具。否则，他什么也不会做。请注意，所有毛绒玩具的转移都是同时进行的。**在这个版本中，如果任何一只蜘蛛在任何时候拥有的毛绒玩具超过 1 ，那么除了 1 之外的所有毛绒玩具都会被扔掉。**

如果每只蜘蛛在当年（当年交换前）拥有的毛绒玩具数量与上一年（上一年交换前）相同，则该过程在当年是稳定的。请注意， 1 年不可能是稳定的。

请找出进程趋于稳定的第一年。

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($2 \le n \le 2 \cdot 10^5$) — the number of spiders.

The following line contains n integers $r_1, r_2, \ldots, r_n$ ($1 \le r_i \le n, r_i \ne i$) — the recipient of the plushie of each spider.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

第一行包含一个整数 t ($1 \le t \le 10^4$) - 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($2 \le n \le 2 \cdot 10^5$) --蜘蛛数量。

下一行包含 n 个整数 $r_1, r_2, \ldots, r_n$ ($1 \le r_i \le n, r_i \ne i$) --每个蜘蛛的毛绒玩具的收件人。

保证所有测试用例中 n 的总和不超过 $2 \cdot 10^5$ 。

Output

For each test case, output an integer on a new line, the first year in which the process becomes stable.

对于每个测试用例，在新行上输出一个整数，即流程趋于稳定的第一年。

Example

Input
5
2
2 1
5
2 3 4 5 1
5
2 1 4 2 3
5
4 1 1 5 4
10
4 3 9 1 6 7 9 10 10 3

Output
2
2
5
4
5

Note

For the second test case:

- At year 1, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Then, year 1's exchange happens.
- At year 2, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Since this array is the same as the previous year, this year is stable.

For the third test case:

- At year 1, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Then, year 1's exchange happens.
- At year 2, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 0]$. Then, year 2's exchange happens. Note that even though two spiders gave spider 2 plushies, spider 2 may only keep one plushie.
- At year 3, the following array shows the number of plushies each spider has: $[1, 1, 0, 1, 0]$. Then, year 3's exchange happens.
- At year 4, the following array shows the number of plushies each spider has: $[1, 1, 0, 0, 0]$. Then, year 4's exchange happens.
- At year 5, the following array shows the number of plushies each spider has: $[1, 1, 0, 0, 0]$. Since this array is the same as the previous year, this year is stable.

对于第二个测试案例：

- 在 1 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量：$[1,1,1,1,1]$.然后，1 年的交换发生了。
- 在 2 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量：$[1,1,1,1,1]$ ：$[1,1,1,1,1]$.由于这个数组与前一年相同，所以这一年是稳定的。

第三个测试案例

- 在第 1 年，下面的数组显示了每只蜘蛛拥有的毛绒玩具数量：$[1,1,1,1,1]$.然后，1 年的交换发生了。
- 在 2 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量：$[1,1,1,1,0]$ ：$[1,1,1,1,0]$.然后，2 年的交换发生了。请注意，即使两只蜘蛛给了蜘蛛 2 毛绒玩具，蜘蛛 2 也只能保留一个毛绒玩具。
- 在 3 年，下面的数组显示了每只蜘蛛拥有的毛绒玩具数量：$[1,1,0,1,0]$.然后，在 3 年进行交换。
- 在 4 年，下面的数组显示了每只蜘蛛拥有的毛绒玩具数量：$[1,1,0,0,0]$ ：$[1,1,0,0,0]$.然后，4 年的交换发生了。
- 在 5 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量：$[1,1,0,0,0]$ ：$[1,1,0,0,0]$.由于这个数组与前一年相同，所以这一年是稳定的。

题解

观察题目给出的样例不难发现，最后的稳定状态是多个蜘蛛间相互传递糖果。

不难想到这一状态对应图里的环。

easy版本可变为求一个有向图中某个点到最近的环的最远距离（当所有的糖都被聚集到环上的时候，达到稳定状态，所以要求最远的距离）。

首先考虑如何找到图中的环。

我们知道入度为0的点一定不在环中，因此我们从入度为0的点开始，沿着有向边删除边，并将新产生的入度为0的点入队。

最后剩下的点入度不为0的即为在环上的点。

接下来只需从这些点出发，逆着有向边的方向跑一遍BFS，即可求出距离。

注意到第一年不可能稳定以及稳定的条件是两年间没有变换，因此答案应为距离+2。

代码如下

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<long long,long long> PLL;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}
```

```
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

void solve(){
    ll n = read();
    vector<ll> indegree(n, 0), dis(n, -1);
    vector<vector<ll>> forward(n), back(n);
    for(ll i=0;i<n;i++){
        ll u = read() - 1;
        forward[i].push_back(u);
        back[u].push_back(i);
        indegree[u] ++;
    }
    queue<ll> q;
    for(ll i=0;i<n;i++){
        if (indegree[i] == 0){
            q.push(i);
        }
    }
    while(!q.empty()){
        ll u = q.front();
        q.pop();
        for(auto v:forward[u]){
            indegree[v] --;
            if (indegree[v] == 0){
                q.push(v);
            }
        }
    }
    queue<PLL> qs;
    for(ll i=0;i<n;i++){
        if (indegree[i]){
            dis[i] = 0;
            for(auto v:back[i]){
                if (!indegree[v]&&dis[v]==-1){
                    dis[v] = 1;
                    qs.push(PLL{v, dis[v]});
                }
            }
        }
    }
    while(!qs.empty()){
        auto t = qs.front();
        ll u = t.first;
        ll tdis = t.second;
        qs.pop();
        for(auto v:back[u]){
            if (dis[v]==-1){
                dis[v] = tdis+1;
                qs.push({v, dis[v]});
            }
        }
    }
    ll depth=0;
```

```
    for(ll i=0;i<n;i++){
        depth = max(depth, dis[i]);
    }
    print(depth + 2);
}

int main(){
    ll t = read();
    while (t--){
        solve();
    }
}
```

G2. Medium Demon Problem (hard version)

time limit per test: 2 second

memory limit per test: 256 megabytes

Difficulty:1900

原题

This is the hard version of the problem. The key difference between the two versions is highlighted in bold.

A group of n spiders has come together to exchange plushies. Initially, each spider has 1 plushie. Every year, if spider i has at least one plushie, he will give exactly one plushie to spider r_i. Otherwise, he will do nothing. Note that all plushie transfers happen at the same time. **In this version, each spider is allowed to have more than 1 plushie at any point in time.**

The process is stable in the current year if each spider has the same number of plushies (before the current year's exchange) as he did the previous year (before the previous year's exchange). Note that year 1 can never be stable.

Find the first year in which the process becomes stable.

这是问题的困难版本。两个版本的主要区别以粗体标出。

一群 n 蜘蛛聚在一起交换毛绒玩具。最初，每只蜘蛛都有 1 个毛绒玩具。每年，如果蜘蛛 i 至少有一个毛绒玩具，他就会给蜘蛛 r_i 一个毛绒玩具。否则，他什么也不会做。请注意，所有毛绒玩具的转移都是同时进行的。**在这个版本中，每只蜘蛛在任何时候都可以拥有一个以上的毛绒玩具。**

如果每只蜘蛛在当年（当年交换前）拥有的毛绒玩具数量与上一年（上一年交换前）相同，则该过程在当年是稳定的。请注意， 1 年不可能是稳定的。

请找出进程趋于稳定的第一年。

Input

The first line contains an integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($2 \le n \le 2 \cdot 10^5$) — the number of spiders.

The following line contains n integers $r_1, r_2, \ldots, r_n$ ($1 \le r_i \le n, r_i \ne i$) — the recipient of the plushie of each spider.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

第一行包含一个整数 t ($1 \le t \le 10^4$) - 测试用例的数量。

每个测试用例的第一行包含一个整数 n ($2 \le n \le 2 \cdot 10^5$) --蜘蛛数量。

下一行包含 n 个整数 $r_1, r_2, \ldots, r_n$ ($1 \le r_i \le n, r_i \ne i$) --每个蜘蛛的毛绒玩具的收件人。

保证所有测试用例中 n 的总和不超过 $2 \cdot 10^5$ 。

Output

For each test case, output an integer on a new line, the first year in which the process becomes stable.

对于每个测试用例，在新行上输出一个整数，即流程趋于稳定的第一年。

Example

Input
5
2
2 1
5
2 3 4 5 1
5
2 1 4 2 3
5
4 1 1 5 4
10
4 3 9 1 6 7 9 10 10 3

Output
2
2
5
5
5

Note

For the second test case:

- At year 1, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Then, year 1's exchange happens.
- At year 2, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Since this array is the same as the previous year, this year is stable.

For the third test case:

- At year 1, the following array shows the number of plushies each spider has: $[1, 1, 1, 1, 1]$. Then, year 1's exchange happens.
- At year 2, the following array shows the number of plushies each spider has: $[1, 2, 1, 1, 0]$. Then, year 2's exchange happens.

- At year 3, the following array shows the number of plushies each spider has: $[1, 3, 0, 1, 0]$. Then, year 3's exchange happens.
- At year 4, the following array shows the number of plushies each spider has: $[1, 4, 0, 0, 0]$. Then, year 4's exchange happens.
- At year 5, the following array shows the number of plushies each spider has: $[1, 4, 0, 0, 0]$. Since this array is the same as the previous year, this year is stable.

对于第二个测试案例：

- 在 1 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量： $[1, 1, 1, 1, 1]$.然后， 1 年的交换发生了。
- 在 2 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量： $[1, 1, 1, 1, 1]$ ： $[1, 1, 1, 1, 1]$.由于这个数组与前一年相同，所以这一年是稳定的。

第三个测试案例

- 在第 1 年，下面的数组显示了每只蜘蛛拥有的毛绒玩具数量： $[1, 1, 1, 1, 1]$.然后， 1 年的交换发生了。
- 在 2 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量： $[1, 2, 1, 1, 0]$ ： $[1, 2, 1, 1, 0]$.然后， 2 年的交换发生了。
- 在 3 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量： $[1, 3, 0, 1, 0]$ ： $[1, 3, 0, 1, 0]$.然后， 3 年的交换发生了。
- 在 4 年，以下数组显示了每只蜘蛛拥有的毛绒玩具数量： $[1, 4, 0, 0, 0]$ ： $[1, 4, 0, 0, 0]$.然后， 4 年的交换发生了。
- 在 5 年，下面的数组显示了每只蜘蛛拥有的毛绒玩具数量： d .由于这个数组与前一年相同，所以这一年是稳定的。

题解

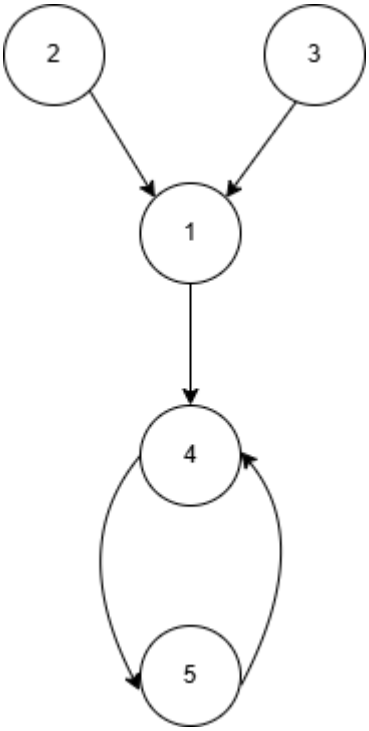
和easy版本一样，所有糖果最终都会聚集到环上。

不同的是，hard版本要计算的是在环上的子树的节点数量的最大值。

下面说明这一点

考虑这个样例
5
4 1 1 5 4

画成图为



不难发现，最终的稳定状态是1， 2， 3节点上的糖果全部转移到环上去。

对于每个子树， 每次有且仅有一个糖果转移到环上去， 转移的时间即为子树的大小。

一棵树的大小可由所有子树的大小的和$+1$得出。dfs可以求解。

答案为最大大小$+2$。

代码如下

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<long long,long long> PLL;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

void solve(){
    ll n = read();
    vector<ll> indegree(n, 0);
    vector<vector<ll>> forward(n), back(n);
    for(ll i=0;i<n;i++){
        ll u = read() - 1;
        forward[i].push_back(u);
```

```
        back[u].push_back(i);
        indegree[u] ++;
    }
    queue<ll> q;
    for(ll i=0;i<n;i++){
        if (indegree[i] == 0){
            q.push(i);
        }
    }
    while(!q.empty()){
        ll u = q.front();
        q.pop();
        for(auto v:forward[u]){
            indegree[v] --;
            if (indegree[v] == 0){
                q.push(v);
            }
        }
    }
    vector<ll> vis(n, 0), size(n, -1);
    function<ll(ll)> dfs = [&](ll u){
        ll cur = 1;
        vis[u] = 1;
        for(auto v:back[u]){
            if (!indegree[v]){
                if (vis[v]) cur += size[v];
                else cur += dfs(v);
            }
        }
        size[u] = cur;
        return cur;
    };
    for (ll i=0;i<n;i++){
        if (!indegree[i] and !vis[i]){
            dfs(i);
        }
    }
    ll maxsize=0;
    for(ll i=0;i<n;i++){
        maxsize = max(maxsize, size[i]);
    }
    print(maxsize + 2);
}

int main(){
    ll t = read();
    while (t--){
        solve();
    }
}
```

H. Hard Demon Problem

time limit per test: 3.5 second

memory limit per test: 512 megabytes

Difficulty:2100

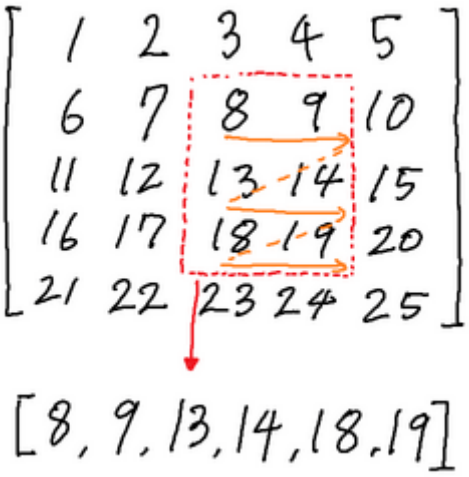
原题

Swing is opening a pancake factory! A good pancake factory must be good at flattening things, so Swing is going to test his new equipment on 2D matrices.

Swing is given an $n \times n$ matrix M containing positive integers. He has q queries to ask you.

For each query, he gives you four integers x_1, y_1, x_2, y_2 and asks you to flatten the submatrix bounded by (x_1, y_1) and (x_2, y_2) into an array A. Formally,
$A = [M_{(x1,y1)}, M_{(x1,y1+1)}, \ldots, M_{(x1,y2)}, M_{(x1+1,y1)}, M_{(x1+1,y1+1)}, \ldots, M_{(x2,y2)}]$.

The following image depicts the flattening of a submatrix bounded by the red dotted lines. The orange arrows denote the direction that the elements of the submatrix are appended to the back of A, and A is shown at the bottom of the image.



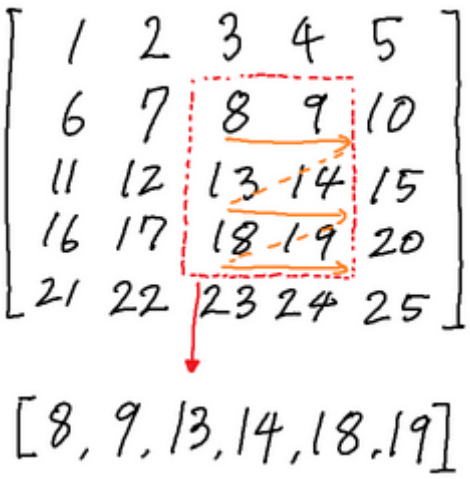
Afterwards, he asks you for the value of $\sum_{i=1}^{|A|} A_i \cdot i$ (sum of $A_i \cdot i$ over all i).

斯温要开一家煎饼厂！一家好的煎饼厂必须擅长把东西压平，所以 Swing 打算在二维矩阵上测试他的新设备。

Swing 给出了一个包含正整数的 $n \times n$ 矩阵 M 。他有 q 个问题要问你。

对于每个问题，他都会给出四个整数 x_1、y_1、x_2、 y_2 ，并要求你将以 (x_1, y_1) 和 (x_2, y_2) 为界的子矩阵平铺到数组 A 中。形式上，
$A = [M_{(x1,y1)}, M_{(x1,y1+1)}, \ldots, M_{(x1,y2)}, M_{(x1+1,y1)}, M_{(x1+1,y1+1)}, \ldots, M_{(x2,y2)}]$.

下图描述了以红色虚线为边界的子矩阵的扁平化过程。橙色箭头表示子矩阵的元素追加到 A 后面的方向， A 显示在图像的底部。



之后，他问你 $\sum_{i=1}^{|A|} A_i \cdot i$ 的值($A_i \cdot i$ 对所有 i 之和)。

Input

The first line contains an integer t ($1 \le t \le 10^3$) — the number of test cases.

The first line of each test contains two integers n and q ($1 \le n \le 2000, 1 \le q \le 10^6$) — the length of M and the number of queries.

The following n lines contain n integers each, the i'th of which contains $M_{(i,1)}, M_{(i,2)}, \ldots, M_{(i,n)}$ ($1 \le M_{(i,j)} \le 10^6$).

The following q lines contain four integers x_1, y_1, x_2, and y_2 ($1 \le x_1 \le x_2 \le n, 1 \le y_1 \le y_2 \le n$) — the bounds of the query.

It is guaranteed that the sum of n over all test cases does not exceed 2000 and the sum of q over all test cases does not exceed 10^6.

第一行包含一个整数 t ($1 \le t \le 10^3$) - 测试用例的数量。

每个测试的第一行包含两个整数 n 和 q ($1 \le n \le 2000, 1 \le q \le 10^6$) - M 的长度和查询次数。

接下来的 n 行分别包含 n 个整数，其中第 i 行包含 $M_{(i,1)}, M_{(i,2)}, \ldots, M_{(i,n)}$ ($1 \le M_{(i,j)} \le 10^6$)。

下面的 q 行包含四个整数 x_1 、 y_1 、 x_2 和 y_2 ($1 \le x_1 \le x_2 \le n, 1 \le y_1 \le y_2 \le n$)--查询的边界。

保证所有测试用例中 n 的总和不超过 2000 ，所有测试用例中 q 的总和不超过 10^6 。

Output

For each test case, output the results of the q queries on a new line.

对于每个测试用例，在新行中输出 q 查询的结果。

Example

Input
2
4 3
1 5 2 4
4 9 5 3
4 5 2 3
1 5 5 2

1 1 4 4
2 2 3 3
1 2 4 3
3 3
1 2 3
4 5 6
7 8 9
1 1 1 3
1 3 3 3
2 2 2 2

Output
500 42 168
14 42 5

Note

In the second query of the first test case, $A = [9, 5, 5, 2]$. Therefore, the sum is $1 \cdot 9 + 2 \cdot 5 + 3 \cdot 5 + 4 \cdot 2 = 42$.

在第一个测试用例的第二个查询中， $A = [9, 5, 5, 2]$ 。因此，总和为 $1 \cdot 9 + 2 \cdot 5 + 3 \cdot 5 + 4 \cdot 2 = 42$

题解

以下分析中$M_{x,y}$中x, y从0开始， $x1, y1, x2, y2y$以及预处理中乘的x, y与题目一样从1开始，求和时为方便书写范围时从1开始（这样与代码基本保持一样）(例如$\sum_{x=x1}^{x2} \sum_{y=y1}^{y2} y \cdot M_{x,y}$中范围和$y$从$1$开始，而求和式中的$M_{x,y}$从0开始)

先考虑$x1 = 1, y1 = 1$时的答案。

对于 A_i , 即 $M_{x,y}$, 有 $i = x \cdot (y_2 - y_1 + 1) + y + 1$

则 $\sum_{i=1}^{|A|} A_i \cdot i$ 可变化为 $\sum_{x=x1}^{x2} \sum_{y=y1}^{y2} (x \cdot (y_2 - y_1 + 1) + y + 1) \cdot M_{x,y}$

我们可以预处理所有的$\sum_{x=1}^{x2} \sum_{y=1}^{y2} x \cdot M_{x,y}, \sum_{x=1}^{x2} \sum_{y=1}^{y2} y \cdot M_{x,y}, \sum_{x=1}^{x2} \sum_{y=1}^{y2} M_{x,y}$ (分别设为 $Sx_{x,y}, Sy_{x,y}, S_{x,y}$

下面考虑$x1, y1$任意时的答案。

按照上面的讨论我们可以得到从$(1, 1)$开始编号的答案，我们考虑我们多加的部分该如何减去。

首先考虑减去周围不该被计入的部分。

以$S_{x,y}$为例， 有 $\sum_{x=x1}^{x2} \sum_{y=y1}^{y2} M_{x,y} = S_{x2,y2} - S_{x1,y2} - S_{x2,y1} + S_{x1,y1}$

再考虑编号变大导致多加的部分

令$x^{'} = x - x1, y^{'} = y - y1$

现在得到的值的表达式为 $\sum_{x=x1}^{x2} \sum_{y=y1}^{y2} M_{x,y} + \sum_{x=x1}^{x2} \sum_{y=y1}^{y2} x \cdot (y_2 - y_1 + 1) \cdot M_{x,y} + \sum_{x=x1}^{x2} \sum_{y=y1}^{y2} y \cdot M_{x,y}$ \textcircled1

而我们实际应该得到的是
$\sum_{x=x1}^{x2}\sum_{y=y1}^{y2} M_{x,y} + \sum_{x=x1}^{x2}\sum_{y=y1}^{y2} \mathbf{x}' \cdot (y_2 - y_1 + 1) \cdot M_{x,y} + \sum_{x=x1}^{x2}\sum_{y=y1}^{y2} \mathbf{y}' \cdot M_{x,y}$
$\textcircled{2}$（注意加粗的部分）

带入可得
$\textcircled{2} = \textcircled{1} - (y_2 - y_1 + 1) \cdot x_1 \cdot \sum_{x=x1}^{x2}\sum_{y=y1}^{y2} M_{x,y} - y1 \cdot \sum_{x=x1}^{x2}\sum_{y=y1}^{y2}$

代码如下

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void solve(){
    ll n = read(), q = read();
    vector<vector<ll>> matrix(n+1, vector<ll>(n+1)), preSum(n+1, vector<ll>
(n+1)), preSumX(n+1, vector<ll>(n+1)), preSumY(n+1, vector<ll>(n+1));
    for(ll x=0;x<n;x++){
        for(ll y=0;y<n;y++){
            matrix[x][y] = read();
        }
    }
    ll sum1, sum2, sum3, value;
    for(ll x=1;x<=n;x++){
        sum1 = 0, sum2 = 0, sum3 = 0;
        for(ll y=1;y<=n;y++){
            value  = matrix[x-1][y-1];
            sum1 += value;
            sum2 += value * x;
            sum3 += value * y;
            preSum[x][y] = preSum[x-1][y] + sum1;
            preSumX[x][y] = preSumX[x-1][y] + sum2;
            preSumY[x][y] = preSumY[x-1][y] + sum3;
        }
    }
    ll x1, y1, x2, y2, s, sx, sy, width;
    vector<ll> res;
    while(q--){
        x1 = read(), y1 = read(), x2 = read(), y2 = read();
        s = preSum[x2][y2] - preSum[x1-1][y2] - preSum[x2][y1-1] + preSum[x1-1]
[y1-1];
        sx = preSumX[x2][y2] - preSumX[x1-1][y2] - preSumX[x2][y1-1] +
preSumX[x1-1][y1-1];
```

```
        sy = preSumY[x2][y2] - preSumY[x1-1][y2] - preSumY[x2][y1-1] +
preSumY[x1-1][y1-1];
        width = y2 - y1 + 1;
        res.push_back(width * sx + sy + s - x1*width*s - y1*s);
    }
    for(auto tres:res){
        pt(tres), putchar(' ');
    }
    puts("");
}

int main(){
    ll t = read();
    while(t--){
        solve();
    }
}
```

拼尽全力终于战胜！ (2024/12/24 18:57)

我的提交

#	提交时间	提交者	问题	语言	判题状态	时间	内存
298170225	Dec/24/2024 18:57 UTC+8	FZANOTFOUND	H - Hard Demon 问题	C++20 (GCC 13-64)	Accepted	1921 ms	144100 KB
298120719	Dec/24/2024 08:15 UTC+8	FZANOTFOUND	G2 - Medium Demon 问题 (hard version)	C++20 (GCC 13-64)	Accepted	171 ms	28300 KB
297701203	Dec/22/2024 01:20 UTC+8	FZANOTFOUND	F - Easy Demon 问题	C++20 (GCC 13-64)	Accepted	421 ms	2600 KB
297701012	Dec/22/2024 01:19 UTC+8	FZANOTFOUND	F - Easy Demon 问题	C++20 (GCC 13-64)	Compilation error	0 ms	0 KB
297699704	Dec/22/2024 01:09 UTC+8	FZANOTFOUND	F - Easy Demon 问题	C++20 (GCC 13-64)	Time limit exceeded on test 25	4000 ms	30900 KB
297699266	Dec/22/2024 01:06 UTC+8	FZANOTFOUND	F - Easy Demon 问题	C++20 (GCC 13-64)	Time limit exceeded on test 35	4000 ms	30600 KB
297697959	Dec/22/2024 00:55 UTC+8	FZANOTFOUND	F - Easy Demon 问题	PyPy 3-64	Time limit exceeded on test 12	4000 ms	38100 KB
297071683	Dec/18/2024 16:13 UTC+8	FZANOTFOUND	G1 - Medium Demon 问题 (easy version)	C++20 (GCC 13-64)	Accepted	218 ms	18700 KB
297071282	Dec/18/2024 16:10 UTC+8	FZANOTFOUND	G1 - Medium Demon 问题 (easy version)	C++20 (GCC 13-64)	Accepted	171 ms	18900 KB
297070958	Dec/18/2024 16:07 UTC+8	FZANOTFOUND	G1 - Medium Demon 问题 (easy version)	C++20 (GCC 13-64)	Accepted	186 ms	19000 KB
296908326	Dec/17/2024 08:46 UTC+8	FZANOTFOUND	E - Insane 问题	PyPy 3-64	Accepted	202 ms	5700 KB
296825098	Dec/16/2024 18:56 UTC+8	FZANOTFOUND	D - Harder 问题	PyPy 3-64	Accepted	296 ms	29800 KB

My contest submissions

#	提交时间	提交者	问题	语言	判题状态	时间	内存
296721888	Dec/16/2024 00:14 UTC+8	FZANOTFOUND	E - Insane 问题	PyPy 3-64	Accepted	234 ms	6600 KB
296701571	Dec/15/2024 23:47 UTC+8	FZANOTFOUND	D - Harder 问题	PyPy 3-64	Time limit exceeded on test 8	2000 ms	32500 KB
296699554	Dec/15/2024 23:45 UTC+8	FZANOTFOUND	D - Harder 问题	PyPy 3-64	Wrong answer on test 1	62 ms	0 KB
296674014	Dec/15/2024 23:20 UTC+8	FZANOTFOUND	D - Harder 问题	PyPy 3-64	Wrong answer on test 2	249 ms	5700 KB
296635421	Dec/15/2024 22:53 UTC+8	FZANOTFOUND	C - Hard 问题	PyPy 3-64	Accepted	218 ms	6400 KB
296626038	Dec/15/2024 22:49 UTC+8	FZANOTFOUND	B - Normal 问题	PyPy 3-64	Accepted	125 ms	2700 KB
296617449	Dec/15/2024 22:45 UTC+8	FZANOTFOUND	A - Easy 问题	PyPy 3-64	Accepted	78 ms	0 KB